

4. Módulos

4.1. Tipos de módulos

Los programas de JavaScript tradicionalmente se han cargado a través de ficheros externos mediante etiquetas **<script>** en el navegador. En los inicios, dichos ficheros definían un conjunto de funciones y variables que se incorporaban **directamente al objeto global**. Esta situación provocaba los siguientes inconvenientes:

- Se podían producir **conflictos** entre nombres de funciones o variables definidas con el mismo nombre en distintos archivos. Esto se solucionaba incorporando un **prefijo** a esos nombres para evitarlo.
- Se podían **modificar** las variables y funciones definidas en otros ficheros, intencionadamente o no, con el consiguiente problema de seguridad que acarreaba.

Para evitar estos problemas, se desarrolló el concepto de **módulo**. Un módulo es un **fichero JavaScript** que permite definir **qué objetos (en forma de variables o funciones) se exportarán** para que puedan ser utilizados por el objeto global u otros módulos. El resto permanecerá oculto y no será modificable desde el exterior.

Se han desarrollado distintos **sistemas de módulos**. A continuación, se indican algunos de ellos:

- **CommonJS**. Es el que se definió para NodeJS.
- **AMD**. Utilizado en navegadores por librerías de carga de módulos.
- **UMD**. Sistema universal compatible con **CommonJS** y **AMD**.
- **ES6**. Sistema de módulos **incorporado al estándar EcmaScript**. Es el sistema llamado a reemplazar a todos los demás. Es compatible con los navegadores actuales. En NodeJS se puede utilizar si los archivos utilizan la extensión **.mjs** en lugar de **.js**.

En este apartado estudiaremos los módulos **ES6**, a los que nos referiremos simplemente como **módulos**.

! IMPORTANTE

Dado que los módulos **ES6** son de reciente incorporación al lenguaje, todavía es posible encontrar gran cantidad de librerías que utilizan alguno de los otros sistemas no estándar.

4.2. *Import y export*

El sistema de módulos define dos directivas en el lenguaje: **import** y **export**:

- **import** se utiliza para importar funcionalidad desde otros módulos. Solo se podrán importar objetos de un módulo si han sido exportadas por este.
- **export** se utiliza para definir qué objetos de los definidos en el módulo serán **accesibles desde el exterior**, es decir, podrán ser importados por el código que utilice dicho módulo.

A continuación, se muestra el código de un módulo, guardado en el archivo `miLibreria.js`:

```
// Fichero miLibreria.js

function saludar(usuario) {
  console.log(`Hola, ${usuario}`);
}

function secreto() {
  console.log("Sssssh... es un secreto...");
}

function despedir(usuario) {
  secreto();
  console.log(`Adiós, ${usuario}`);
}

// Exportación de objetos.
// Se exporta un objeto con dos propiedades.
// Cada propiedad hace referencia a una de las funciones definidas arriba.
// La función 'secreto' no se exporta, por lo que no es visible fuera del
módulo;
// sin embargo, dentro del módulo sí que puede utilizarse.
export {
  saludar,
  despedir
}
```

Seguidamente, se muestra el código de un programa, almacenado en el fichero `principal.js`, que utiliza el módulo definido anteriormente:

```
// Fichero principal.js

// Importación de las funciones exportadas por el módulo
// Mediante esta sintaxis se pueden indicar las funciones que se deseen
importar,
// que no tienen por qué ser todas
import {saludar, despedir} from './miLibreria.js';

// A partir de aquí se pueden utilizar las funciones importadas, 'saludar'
y 'despedir'
saludar("Juan"); // Hola, Juan
despedir();      // "Sssssh... es un secreto...
                // Adiós, Juan
```

Si se desea importar toda la funcionalidad de un módulo, es posible utilizar la siguiente sintaxis:

```
// Fichero principal.js

import * as miLibreria from './miLibreria.js';
// El objeto 'miLibreria' tiene definidas todas las funciones exportadas por
la librería

miLibreria.saludar("Juan"); // Hola, Juan
miLibreria.despedir();      // "Sssssh... es un secreto...
                          // Adiós, Juan
```

! IMPORTANTE

Existen otras posibilidades para exportar objetos utilizando módulos, como la opción `default`.

4.3. Módulos en HTML

Para trabajar con módulos en HTML y utilizar `import` para cargarlos en nuestros programas es necesario utilizar el atributo `type="module"` en la etiqueta `<script>`.

```
<script type="module">
  import * as miLibreria from './miLibreria.js';

  miLibreria.saludar("Juan");
  miLibreria.despedir();
</script>
```

! IMPORTANTE

`import` siempre debe definir una ruta (absoluta o relativa) al archivo del módulo que se pretenda importar: si el archivo `miLibreria.js` se encuentra en la misma carpeta que la página HTML se debe indicar `./` para señalar la ruta al directorio actual; el código `import * as miLibreria from 'mi-Libreria.js'` no funcionará.

! IMPORTANTE

¡Muy importante! Los módulos solo se cargan a través de HTTP, no a través del sistema de ficheros. Por tanto, para hacer pruebas no es posible lanzar la página haciendo doble clic sobre el fichero HTML, sino que se debe cargar a través de un servidor web (por ejemplo, usando la extensión **Live Server** de Visual Studio Code).

Recuerda que, si la barra de direcciones comienza por `file://` al cargar una página web, estás realizando la carga a través del sistema de ficheros.

4.4. Librerías externas: CDN

Mediante el sistema de módulos, podemos acceder a un gran número de **librerías externas** creadas por otras personas. JavaScript tiene una de las comunidades de desarrolladores más grandes del mundo, por lo que la oferta es muy extensa.

La mayoría de las librerías más utilizadas ofrecen compatibilidad con `import`, aunque no todas: es posible que algunas de ellas nos pidan que las utilicemos cargándolas en el objeto global mediante una etiqueta `<script>` convencional.

En cuanto a la modalidad de carga, nos encontraremos con dos posibilidades:

- **Descarga** del fichero (o ficheros) para almacenarlos junto con nuestro código.
- **Acceso** al fichero (o ficheros) a través de una **URL** sin necesidad de descargar. En este caso, la URL suele apuntar a un **CDN**.

Un **CDN**, o *content delivery network*, es un **servicio distribuido de alojamiento de ficheros**. Su principal función es **garantizar la descarga de dichos ficheros de manera óptima**. Para ello, utiliza una arquitectura distribuida de servidores: la descarga, que se produce de manera transparente al usuario, se realiza desde el servidor que ofrezca mejor servicio, bien por distancia geográfica, por latencia o por carga actual del sistema. Encontrarás algunos de los CDN más famosos en los enlaces web al final de la unidad.

La elección de utilizar un CDN frente a descargar la librería para almacenarla en nuestro servidor tiene una serie de **ventajas**:

- Ahorro de espacio.
- Ahorro de ancho de banda en nuestro servidor.
- Mejor comportamiento en la descarga.
- Simplicidad a la hora de realizar actualizaciones de versiones.

También tiene desventajas, como la **dependencia de un servidor externo**: si el CDN se cae, o deja de dar servicio, perderemos el acceso a las librerías.